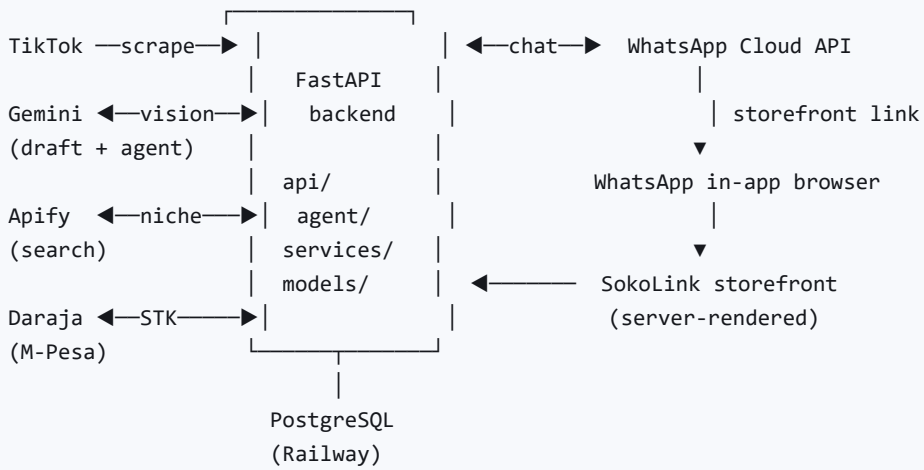
TECHNICAL DOCUMENT

SokoLink — Architecture & Tech Stack

Python · FastAPI · Postgres · Gemini · WhatsApp Cloud API · M-Pesa Daraja

Version 1.0 · August 2026 · Owner: Fredrick Muchoya

1. System overview



One FastAPI application serves three surfaces: the WhatsApp webhook, the server-rendered storefront, and the internal APIs behind both. A single Postgres database is the source of truth.

2. Tech stack

LAYER	CHOICE	WHY THIS
Language	**Python 3.11+**	The maintainer's strongest language. On a solo build, ability to debug unaided outweighs framework elegance.
Web framework	**FastAPI**	Async, typed, small. Every endpoint is visible; nothing is hidden behind conventions.
Validation	**Pydantic v2**	One tool for API schemas *and* LLM output schemas. The single most important guardrail in the system.
Database	**PostgreSQL** (Railway)	Relational data with hard constraints. The DB enforces rails the application cannot bypass.
ORM / migrations	**SQLAlchemy 2.0 + Alembic**	Migrations are schema version control. Every change is reviewed before it is applied.

Templating	**Jinja2**	Server-rendered storefront. Minimal client JS on the buyer's critical path.
AI	**Gemini** via `google-genai`	Tested best on Sheng/Swahili, both printed on covers and spoken in clips. Called directly — no wrapper.
Scraping	**Apify**, behind our own adapter	Profile pulls without seller OAuth. The adapter means swapping engines is a one-file change.
Messaging	**WhatsApp Cloud API**	The interface Kenyan commerce already lives in.
Payments	**M-Pesa Daraja** (STK push)	Universal in Kenya, including for the unbanked.
Testing	**pytest**	Transactional fixtures against real Postgres; all external services mocked.
Hosting	**Railway**	App and database on one platform.

> **No AI framework.** No LangChain, no vector database, no orchestration layer. Provider SDKs are called directly with Pydantic schemas as the guardrail. Frameworks hide the loop, the tool call, the retry — exactly the mechanics you need visible when something breaks at 11pm.

3. Repository layout

```
backend/
  app/
    main.py          thin — wires routers, owns nothing
    config.py        typed settings via pydantic-settings
    db.py            engine + session; pool_pre_ping for cloud DBs
    models/          SQLAlchemy models — the DB rails live here
    schemas/         Pydantic wire + LLM schemas
    services/        business logic (scraper, storefront, orders, mpesa)
    agent/           LLM-facing code (draft, sales)
    api/             HTTP routes — thin, delegate to services
    templates/       Jinja2 — the storefront
  alembic/           migrations, reviewed line by line
  tests/             pytest, mirrors app/
  spikes/            throwaway probes against real APIs
docs/                plan, concepts, this document
```

The layering rule: `api/` handles HTTP shapes, `services/` holds business logic, `models/` owns persistence. When a file in `main.py` grows past ~100 lines, something is in the wrong place.

4. Data model

MODEL	PURPOSE	RAILS ENFORCED IN THE DATABASE
`Account`	Seller login	Unique email/phone; Argon2id hash never leaves the row
`Seller`	Storefront identity	Unique handle; one-to-one with account
`Product`	A catalogue item from one video	`stock >= 0`; unique `tiktok_video_id`; **published requires a price**
`Order`	A purchase and its payment state	State machine; unique checkout request id
`Customer`	A buyer, keyed by phone	Unique per seller
`NicheSearch` / `Hook`	Soko Intel corpus	Cached by keyword; hooks tagged by niche and language register

> **Money is stored as integer KES.** No float or decimal price columns anywhere. A rounding error in a price is a rounding error in what a buyer is charged.

5. The price extraction cascade

Discovery finding, verified against 24 real captions: **captions carry no prices.** Zero mention KSh; only three contain any price-like number. The product must be read from the media, not the text.

```
Tier 1  caption + hashtags       near-free   → category hints, sometimes sizes
   │    (rarely yields a price)
   ▼
Tier 2  cover image                cheap       → price printed as text overlay
   │    Gemini vision
   ▼
Tier 3  video: visual + AUDIO      expensive   → seller says the price in Sheng
        Gemini multimodal                        highest yield
```

Each tier escalates only when the one above fails to produce a confident price. **Every video is processed once ever**, keyed by TikTok video id, and cached — per-request processing would be ruinous.

Proven live on real clips: KES 500 / 550 / 550 / 600 read correctly from spoken and on-screen prices.

6. LLM integration patterns

6.1 Structured output, not parsed hope

We do not ask the model for JSON and pray. A Pydantic schema is passed to the API, and generation is *constrained* so the output can only be a valid instance of that shape — no prose, no fenced code blocks, no missing keys.

```
cover image + hashtags ──▶ Gemini (schema=ProductDraft) ──▶ validated object
                           constrained decoding
```

6.2 The agent proposes, code disposes

> The LLM output is always a **proposal**, never a decision, on anything that matters. The agent drafts words and reads a price it can see. **Prices, stock, payment status and consent are decided by plain code with hard rules.**
>
> A misread price sits in a draft the seller reviews. Publishing is a deliberate human action that still requires a price. The AI removes typing; it cannot sell anything.

6.3 No RAG

RAG solves retrieval-by-similarity when you cannot predict which document is relevant. Every lookup here is by a known key — this product, this seller, this niche. Retrieval-by-known-key is a `SELECT`. Do not bring a vector database to a `WHERE id = ?` problem.

The one place similarity may genuinely arrive: "find hooks similar in spirit to this product." Recognise it if it comes; do not reach for it early.

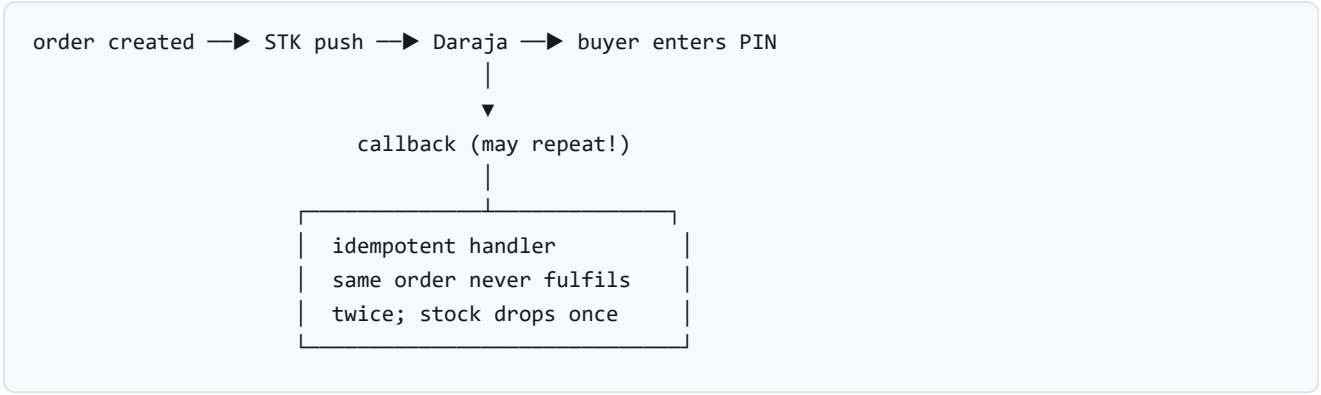
7. The WhatsApp storefront flow

Rather than redirecting buyers to a separate website, SokoLink launches a mobile-first web experience directly from WhatsApp. DESIGN NOTE This uses the ordinary in-app browser, not WhatsApp Flows — which removes any dependency on Flows approval and gives full control of the interface.

The three hard parts

PROBLEM	APPROACH
Identity across the hop. The storefront must know which buyer and which shop, with no login.	The chat mints a **signed, scoped, short-lived token** in the link. CRITICAL Never put a phone number in a URL — links leak through history, referrer headers, and buyers forwarding them to friends.
M-Pesa interrupts the browser. The STK prompt takes over the same phone the webview runs on.	Checkout **polls order status** and renders paid / failed / timed-out honestly when the buyer returns. The callback remains the only truth; the page merely reflects it.
The return hop. A confirmed order should not strand the buyer on a success page.	Deep-link back into the conversation with the seller. Designed, not bolted on.

8. Payment rails



> **Idempotency is not optional.** Daraja retries callbacks, and so does Meta with webhooks. Any handler that is not safe to run twice will eventually double-fulfil an order or double-charge a buyer. Both are tested with explicit replay tests.

Rails before agent. The payment path is built and proven with a plain button before the sales agent is permitted to call it. The agent *requests* a charge; the rails execute it.

9. Cost rails

COST CENTRE	GUARD
Apify — per-seller scraping	At most once per day per seller, cached. Never per request.
Apify — niche search UNBOUNDED	The only cost a user can trigger repeatedly at will. Three guards, all required: **premium tier only** (cost tracks revenue), **per-seat quotas**, **result caching**.
Gemini — video tier	Most expensive tier of the cascade. Once ever per video id. Billing stays enabled — the free tier caps at 20/day and has already bitten once.
Gemini — drafting	On-demand, split out of ingest so ingest stays cheap and fast.

10. Security

- **Secrets** live in `.env` (gitignored) and Railway variables. Never in `.env.example` — that file is committed, and this has already caught us once.
- **Auth** — Argon2id password hashing, signed JWT sessions, anti-enumeration on login.
- **Ownership scoping** — every product query is scoped to the caller's storefront; another seller's rows read as not-found.
- **Webhook verification** — signature checks on both Meta and Daraja callbacks before any state changes.

- **No identity in URLs** — storefront links carry signed, expiring tokens.
- **Rate limiting and input validation** on every public route.

11. Testing strategy

LAYER	WHAT IT COVERS
Unit	Price parsing, phone normalisation, slug generation, pure helpers
Integration	API routes and queries against real Postgres, in rolled-back transactions
Money paths	Every order state transition, plus **explicit replay tests** proving idempotency
AI accuracy	Fixture set of real Sheng captions and covers, guarding prompt changes from silent regression

External services are always mocked. Tests never call Apify, Gemini, Meta or Daraja — those cost money and rate-limit. Tests ship in the same PR as the logic.

12. Documentation standard

Every file opens with a header docstring covering what it does, the pipeline it sits in (as an ASCII diagram when data flows through it), and **why it exists and is shaped that way**. Comments explain *why*, not *what*.

> In six weeks nobody remembers why the cascade escalates in that order, or why a callback is idempotent. The code has to say so itself. This is what makes the codebase understandable, debuggable, and safe to change.